

Exp-3

1.fork()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        printf("Child process, PID = %d\n", getpid());
    } else if (pid > 0) {
        // Parent process
        printf("Parent process, PID = %d\n", getpid());
    } else {
        perror("fork failure");
    }

    return 0;
}
```

2.exit()

```
#include<stdio.h>
#include <unistd.h>
#include<sys/types.h>
#include<stdlib.h>
int main(){
    printf("Exiting with status code 0\n");
    exit(0);
    printf("this wont be running");
}
```

3.exec()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>
int main() {
    printf("before exec()\n");
    execl("/bin/ls", "ls", "-l", NULL);
    perror("exec() failed");
    return 1;
}
```

4.wait()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>
#include <sys/wait.h>

int main() {
    pid_t pid=fork();

    if(pid==0){
        printf("child process\n");
        exit(0);
    }
    else{
        wait(NULL);
        printf("child process finished parent process continuing\n");
    }
    return 0;
}
```

5.getpid()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>
#include <sys/wait.h>

int main() {
    printf("get process id,pid %d\n",getpid());
    return 0;
}
```

6.getuid()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>
#include <sys/wait.h>
```

```
int main() {
    printf("user id:%d\n",getuid());
    return 0;
}
```

7.setuid()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>
#include <sys/wait.h>

int main() {
    if(setuid(1000)==0){
        printf("user id changed to 1000\n");
    }
    else{
        perror("setuid() failed");
    }
    return 0;
}
```

8.sbrk()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>
#include <sys/wait.h>

int main() {
    void *end=sbrk(0);
    printf("program break before %p\n",end);
    brk(end+1024);
    printf("program break after %p\n",sbrk(0));
    return 0;
}
```

9.nice()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>
```

```
#include <sys/wait.h>

int main() {
    int priority=nice(5);
    printf("the new priorit is :%d\n",priority);
    return 0;
}
```

10.sleep()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>
#include <sys/wait.h>

int main() {
    printf(" sleeping for 3\n");
    sleep(3);
    printf("awake now! \n");
    return 0;
}
```

EXP-4

FCFS:

```
def fcfs(processes, burst, arrival):
    n = len(processes)
    start = [0]*n
    waiting = [0]*n
    turnaround = [0]*n

    for i in range(n):
        if i == 0:
            start[i] = arrival[i]
        else:
            start[i] = max(start[i-1] + burst[i-1], arrival[i])
            waiting[i] = start[i] - arrival[i]
            turnaround[i] = waiting[i] + burst[i]

    print("\nP\tWaiting\tTurnaround")
```

```

for i in range(n):
    print(f"{processes[i]}\t{waiting[i]}\t{turnaround[i]}")

print(f"\nAverage Waiting Time: {sum(waiting)/n:.2f}")
print(f"Average Turnaround Time: {sum(turnaround)/n:.2f}")

```

Example

```

processes = [1, 2, 3, 4]
burst = [5, 3, 16, 8]
arrival = [0, 1, 2, 4]

```

```
fcfs(processes, burst, arrival)
```

FJS_NONPREEMTIVE:

```

def sjf_non_preemptive(p, b, a):
    n = len(p)
    w = [0]*n
    t = [0]*n
    time = 0
    done = [False]*n

    for _ in range(n):
        idx = -1
        for i in range(n):
            if not done[i] and a[i] <= time and (idx == -1 or b[i] < b[idx]):
                idx = i
        if idx == -1:
            time += 1
            continue
        w[idx] = time - a[idx]
        time += b[idx]
        t[idx] = w[idx] + b[idx]
        done[idx] = True

    for i in range(n):
        print(f"P{p[i]} Waiting={w[i]} Turnaround={t[i]}")
    print(f"Avg Waiting={sum(w)/n:.2f} Avg Turnaround={sum(t)/n:.2f}")

```

Example

```
sjf_non_preemptive([1,2,3,4], [5,3,16,8], [0,1,2,3])
```

SJF_PREEMTIVE

```

def sjf_preemptive(p, b, a):
    n = len(p)
    r = b.copy()
    w = [0]*n

```

```

t = [0]*n
time = done = 0

while done < n:
    idx = -1
    for i in range(n):
        if a[i] <= time and r[i] > 0 and (idx == -1 or r[i] < r[idx]):
            idx = i
    if idx == -1:
        time += 1
        continue
    r[idx] -= 1
    time += 1
    if r[idx] == 0:
        done += 1
        t[idx] = time - a[idx]
        w[idx] = t[idx] - b[idx]

for i in range(n):
    print(f"P{p[i]} Waiting={w[i]} Turnaround={t[i]}")
print(f"Avg Waiting={sum(w)/n:.2f} Avg Turnaround={sum(t)/n:.2f}")

```

Example

```

sjf_preemptive([1,2,3], [6,8,7], [2,5,1])

```

ROUND_ROBBIN

```

def rr(p, b, a, q):
    n = len(p)
    r = b.copy()
    w = [0]*n
    t = [0]*n
    time = done = 0
    while done < n:
        for i in range(n):
            if a[i] <= time and r[i] > 0:
                k = min(r[i], q)
                r[i] -= k
                time += k
                if r[i] == 0:
                    done += 1
                    t[i] = time - a[i]
                    w[i] = t[i] - b[i]
        else:
            if all(a[i] > time or r[i]==0 for i in range(n)):
                time += 1
    for i in range(n):
        print(f"P{p[i]} W={w[i]} T={t[i]}")

```

```
print(f"Avg W={sum(w)/n:.2f} Avg T={sum(t)/n:.2f}")
```

```
# Example
```

```
rr([1,2,3], [5,4,2], [0,1,2], 2)
```

EXP-5

Single-threading

```
import threading
```

```
import time
```

```
def task(name):
```

```
    for i in range(3):
```

```
        print(f"{name}-step{i}")
```

```
        time.sleep(1)
```

```
task("singel Thread")
```

MULTITHREAD

```
import threading
```

```
import time
```

```
def task(name):
```

```
    for i in range(3):
```

```
        print(f"{name}-step{i}")
```

```
        time.sleep(1)
```

```
t1=threading.Thread(target=task,args=("Thread-1",))
```

```
t2=threading.Thread(target=task,args=("Thread-2",))
```

```
t1.start()
```

```
t2.start()
```

```
t1.join()
```

```
t2.join()
```

```
print("Both threads finished")
```

3. Synchronization between two threads :

```
import threading
```

```
import time
```

```

lock = threading.Lock()
counter = 0

def increment(name):

    global counter
    for _ in range(5):
        time.sleep(0.5)
        with lock:
            counter += 1
            print(f'{name} incremented counter to {counter}')

t1 = threading.Thread(target=increment, args=("Thread-A",))
t2 = threading.Thread(target=increment, args=("Thread-B",))

t1.start()
t2.start()

t1.join()
t2.join()

print("Final counter:", counter)

```

4.Daemon thread execution :

```

import threading
import time

def background_task():
    """This runs in the background"""
    for i in range(5):
        print(f'[Background Task] Step {i}')
        time.sleep(1)
    print("[Background Task] Finished")

def current_task():
    """This runs in the main thread"""
    for i in range(5):
        print(f'[Current Task] Step {i}')
        time.sleep(2)
    print("[Current Task] Finished")

bg_thread = threading.Thread(target=background_task, daemon=True)
bg_thread.start()

current_task()

```



```
bg_thread.join()
print("All tasks finished")
```

5.ThreadPool Execution :

```
import time
from concurrent.futures import ThreadPoolExecutor

def task(name):
    """A simple task that sleeps for 2 seconds."""
    print(f"\nTask {name} started")
    time.sleep(2)
    print(f"\nTask {name} finished")

with ThreadPoolExecutor(max_workers=3) as executor:
    tasks = [executor.submit(task, i) for i in range(5)]

print("All tasks submitted")
```